

A3-PS003-TC001

Federated Traffic-Flow Prediction with Communication-Cost Evaluation

Visionaries
hridyanshisinghal385@gmail.com

AIORI Hackathon 3

October 14, 2026

Problem Description

Why centralised traffic forecasting is a problem, and what this project builds.

- Centralised traffic forecasting needs roadside sensors and cameras to **stream raw data continuously** to a cloud server.
- This consumes **cellular bandwidth** and concentrates **private travel logs** in one place.
- **Federated Learning (FL)** trains models locally and shares only model weight updates with a central server.
- Traffic data is **non-IID**: each intersection has different volumes, peak hours and road types, which can slow or destabilise FedAvg.
- **Goal:** build a federated simulator on a public traffic dataset split across 3–5 virtual intersections. Compare federated and centralised forecasting on accuracy (MAE, RMSE), communication bytes per round and convergence, under IID and non-IID splits.
- **Out of scope:** live cameras, city-wide deployment and formal privacy guarantees.

Background

Key concepts needed to understand the problem and the proposed evaluation.

- **Traffic-flow forecasting:** predict future flow or speed from a window of past readings (e.g. the last 12 steps to predict the next 3 or 12).
- **Federated Averaging (FedAvg):** each round the server sends the global model to the clients; each client trains for E local epochs on its own data and returns its weights; the server averages them, weighted by client sample count [1].
- **Non-IID data:** clients' data distributions differ, which causes client drift and slower convergence.
- **Communication cost:** bytes per round ≈ 2 × (participating clients) × (parameter count) × (bytes per parameter). The factor 2 covers download plus upload; float32 uses 4 bytes per parameter.
- **Why compact models:** edge devices have limited bandwidth and compute, so small models (a few thousand parameters) keep transfer cost low. We compare four such models: GRU, LSTM, 1D-CNN/TCN and MLP.

Literature & RFC Review

Most relevant existing studies and standards only.

Work	Relevance
McMahan et al., 2017 (FedAvg) [1]	Core algorithm; the problem statement's reference
Zhao et al., 2018, *Federated Learning with Non-IID Data* [2]	Shows how accuracy degrades under skewed client data
Li et al., 2020, *FedProx* [3]	Proximal term to stabilise training on heterogeneous clients
Konečný et al., 2016, *Strategies for Improving Communication Efficiency* [4]	Compressing updates to cut bytes per round
Liu et al., 2020, *Privacy-Preserving Traffic Flow Prediction: A Federated Learning Approach* [5]	FL applied to traffic forecasting
Li et al., 2018, DCRNN [6]	Standard METR-LA benchmark and baseline metrics

RFCs / standards: there is no single RFC for FL; it is research-driven. Only general networking context applies (TCP/IP overhead, serialisation formats).

Proposed Solution

Overall design of the federated traffic-forecasting simulator.

1. **Dataset:** one public traffic dataset. METR-LA is a good fit (207 loop sensors, 5-minute readings); PEMS-BAY also works.
2. **Virtual clients:** 5 clients (within the 3–5 limit). *Non-IID split:* group sensors by geographic / traffic-profile cluster so each client sees a different distribution. *IID split:* assign sensors to clients at random.
3. **Preprocessing:** chronological split (70% train, 10% validation, 20% test); normalisation statistics computed on training data only, to avoid leakage.
4. **Models:** four compact forecasters on the same 12-step input window: **GRU** (primary), **LSTM**, **1D-CNN/TCN** and **MLP** (details on the next slide).
5. **Training modes** (for every model): (a) centralised baseline on pooled data, (b) FedAvg with IID clients, (c) FedAvg with non-IID clients.
6. **Metrics:** MAE, RMSE, bytes per round, cumulative bytes, and rounds to reach a target MAE.
7. **Rigor:** every experiment runs with 3 random seeds, reported as mean ± std.

Models Under Evaluation

One primary model plus three comparison models; all are compact and share the same FedAvg setup.

Model	Role	Why it is included	Approx. params → bytes/round*
GRU (32 hidden units)	Primary	Best balance of accuracy and size; trains fast and is stable under FedAvg	~3.5k → ~138 KB
LSTM (32 hidden units)	Comparison	Classic traffic-forecasting model; shows whether its extra memory cell is worth ~30% more bytes	~4.6k → ~183 KB
1D-CNN / TCN (3 causal conv layers, 32 channels)	Comparison	Learns temporal patterns by convolution instead of recurrence; fast and easy to parallelise	~6.4k → ~257 KB
MLP (12 → 64 → 32 → 3)	Lightweight baseline	Cheapest option; sets the minimum-cost floor and shows what accuracy is given up for it	~3.0k → ~120 KB

* Approximate; assumes 1 input feature, a 3-step horizon, 5 clients and float32 (bytes/round = 2 × clients × params × 4). Final values will be measured from the implementation.

Same conditions for all models: identical data splits (IID and non-IID), the same FedAvg loop, seeds 0, 1, 2 and the same metrics, so only the model changes. Results are reported as accuracy (MAE, RMSE) against bytes per round.

Optimization Proposed

Before vs after comparison. Core requirement stays FedAvg; the same optimisations are applied to every model.

Aspect	Before (vanilla FedAvg)	After (proposed)	Measured result
Update precision	float32 (4 B/param)	float16 quantised: about half the bytes per round	[fill]
Local epochs (E)	Fixed, untuned	Tuned, e.g. $E \in \{1, 3, 5\}$, to trade rounds against bytes	[fill]
Non-IID stability	Plain averaging	Optional FedProx term ($\mu > 0$)	[fill]
Convergence	Constant learning rate	Learning-rate decay and early stopping on validation MAE	[fill]
Reporting	Final accuracy only	Accuracy vs cumulative-bytes curve	[fill]

Results: [fill] with your measured MAE, RMSE, bytes and rounds (mean ± std over 3 seeds) for each row and each model.

Solution Architecture & Design

Visual: UML class diagram of the federated simulator.

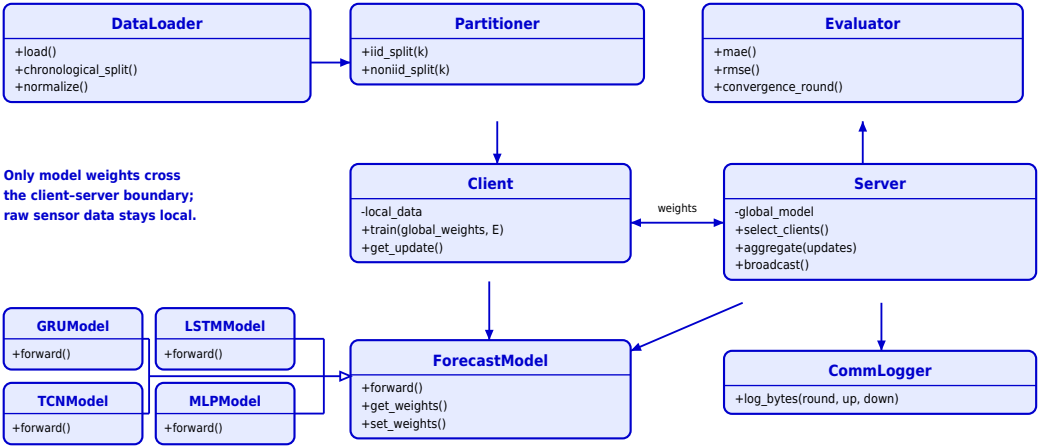


Figure: System Architecture UML Diagram (GRU, LSTM, TCN and MLP models extend ForecastModel)

Data Flow Diagram

Overview of the architecture and data flow between clients and server (repeated for each of the 4 models).

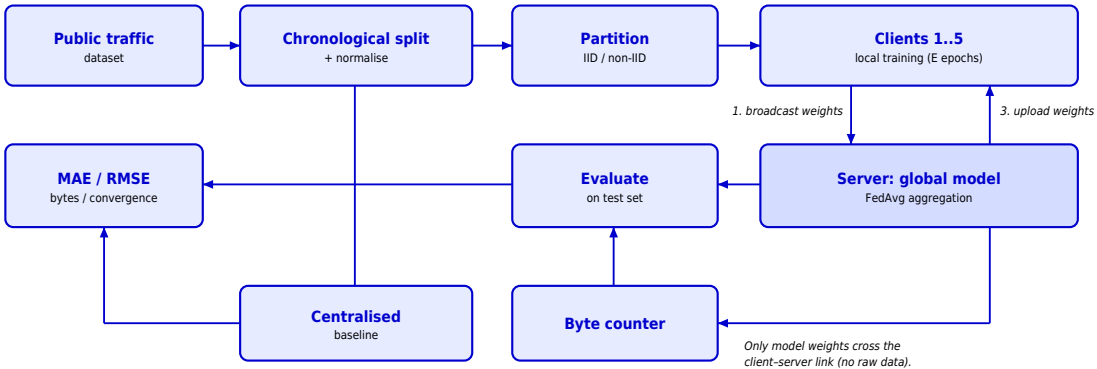


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Deployment milestones and methodology.

Phase	Milestone
1	Set up private GitHub repo and add the aiori-hackathon account; download dataset and run EDA
2	Preprocessing, chronological split, IID and non-IID partitioners
3	Four compact models (GRU, LSTM, 1D-CNN/TCN, MLP) plus centralised baseline
4	FedAvg simulator with byte-count logging
5	IID vs non-IID experiments for each model, 3 seeds each
6	Optimisation experiments (quantisation, E tuning, optional FedProx)
7	Final report, PDF upload to the repo, prototype demo with pseudocode snippet

Methodology: iterative development, fixed seeds (0, 1, 2), and everything version-controlled in the team repository.

Bibliography

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas. “Communication-Efficient Learning of Deep Networks from Decentralized Data.” In: AISTATS, PMLR 54 (2017).

[2] Y. Zhao et al. “Federated Learning with Non-IID Data.” arXiv:1806.00582 (2018).

[3] T. Li et al. “Federated Optimization in Heterogeneous Networks.” In: MLSys (2020).

[4] J. Konečný et al. “Federated Learning: Strategies for Improving Communication Efficiency.” arXiv:1610.05492 (2016).

[5] Y. Liu et al. “Privacy-Preserving Traffic Flow Prediction: A Federated Learning Approach.” In: IEEE Internet of Things Journal (2020).

[6] Y. Li, R. Yu, C. Shahabi, Y. Liu. “Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting.” In: ICLR (2018).

Thank You for Your Attention!